



ichosynth — Ambiente di sviluppo

Guida completa e aggiornata per Windows e macOS

Tutto il necessario per compilare, modificare e caricare il firmware: dalla toolchain alle librerie, fino al primo upload sul Teensy 4.1.

Aggiornata

Giugno 2026

arduino-cli

richiesto

teensy:avr

≥ 1.61.0

OS

Windows · macOS

⚡ **Hai fretta?** ichosynth esegue il firmware reale **TCERN**, e un solo comando lo compila per il nostro hardware: `python teensy/build_toern.py`. Vai al [capitolo 3](#). Il resto della guida spiega ogni passo e prerequisito.

Indice

- 1 · Cosa installiamo
- 2 · Requisiti
- 3 · Via rapida: un comando compila tutto
- 4 · Installare la toolchain (arduino-cli)
- 5 · Aggiungere il supporto Teensy (il core teensy:avr)
- 6 · Le librerie
- 7 · Come funziona lo script di build
- 8 · Compilare e caricare
- 9 · Compilare dall'Arduino IDE (alternativa)
- 10 · Strumenti opzionali (Python, Git)
- 11 · Checklist e problemi comuni

1 · Cosa installiamo

ichosynth è un **Teensy 4.1** che esegue il firmware reale **TCERN** (di SP_ / soundpauli — <https://toern.live>) su componenti economici saldati a mano. Nel repo non teniamo una copia separata di TCERN: uno script di build scarica i sorgenti originali e li compila per il nostro hardware. Ecco cosa serve:

Componente	Versione	A cosa serve	Obbligatorio
arduino-cli	ultima	la toolchain di build da riga di comando	✓
core teensy:avr	≥ 1.61.0	il compilatore Teensy + il Teensy Loader	✓
Python	3.x	esegue <code>teensy/build_toern.py</code> (lo script di build)	✓

Componente	Versione	A cosa serve	Obbligatorio
Git	ultima	lo script clona i sorgenti TCERN alla prima esecuzione	✓
Driver personalizzati	nel repo	i2cEncoderLibV2 , FastTouch , Ichos01ed — già in teensy/libraries/	✓ (inclusi)
GitHub CLI	ultima	opzionale, per versionare il progetto	opzionale

⚙️ **L'unica trappola che frega tutti:** TCERN è un'unica unità di compilazione da ~23k righe, e il gcc del Teensy **va in crash con il -O2 di default** (esce con 0xffffffff — un internal compiler error / out-of-memory). La soluzione è compilare a **-O1** (`opt=o1std`). Lo script di build lo imposta per te; se invece compili dall'Arduino IDE devi scegliere **Tools → Optimize → "Fast"**. Vedi il [capitolo 7](#).

2 · Requisiti

- **Windows:** Windows 10 o 11, 64 bit.
- **macOS:** macOS 10.15 o successivo; supporto nativo **Intel** e **Apple Silicon** (M1/M2/M3/M4).
- **Cavo micro-USB dati** (non solo-carica) per il Teensy 4.1.
- Un **Teensy 4.1 con 16 MB di PSRAM** (entrambi i chip) saldati — TCERN usa la PSRAM per i buffer dei campioni.
- ~1 GB di spazio su disco per toolchain + core + librerie.

3 · Via rapida: un comando compila tutto

Una volta che hai **arduino-cli**, il **core teensy:avr**, **Python** e **Git** (capitoli 4–5), compilare il firmware è un solo comando dalla radice del repo:

```
python teensy/build_toern.py
```

Questo produce il firmware in `teensy/firmware/toern.hex` . Per compilare e flashare in un colpo solo (se `teensy_loader_cli` è nel tuo PATH):

```
python teensy/build_toern.py --flash
```

Flag utili:

Flag	Effetto
<code>--flash</code>	dopo la compilazione, carica con <code>teensy_loader_cli</code>
<code>--keep</code>	mantiene la cartella di build temporanea (per ispezionare i sorgenti preparati/patchati)

💡 La **prima** compilazione è lenta (il core Teensy viene compilato una volta) e inoltre **clona i sorgenti TCERN** in `emulator/toern-src/` se non ci sono ancora. Le compilazioni successive sono rapide.

! Vuoi solo flashare un `.hex` **già pronto**? C'è un flasher GUI a un clic in `_FLASHER/` — vedi il [capitolo 8.3](#).

4 · Installare la toolchain (arduino-cli)

Compiliamo da riga di comando con `arduino-cli`.

Windows

```
winget install ArduinoSA.CLI
```

macOS

```
brew install arduino-cli
```

Verifica che sia nel PATH:

```
arduino-cli version
```

i Driver: su Windows 10/11 e su macOS **non servono driver aggiuntivi** per il Teensy 4.1.

5 · Aggiungere il supporto Teensy (il core teensy:avr)

Il compilatore Teensy e il Teensy Loader arrivano dal **core teensy:avr**. Aggiungi l'indice dei pacchetti di PJRC, aggiorna e installa il core (ci serve la **versione $\geq 1.61.0$**):

```
arduino-cli config init
arduino-cli config add board_manager.additional_urls https://www.pjrc.com/teensy/package_teensy_index.json
arduino-cli core update-index
arduino-cli core install teensy:avr
```

Controlla la versione che hai ottenuto:

```
arduino-cli core list
```

Dovresti vedere `teensy:avr` alla **1.61.0** o successiva.

💡 Preferisci una GUI? In alternativa puoi installare **Teensyduino 1.61** dal Boards Manager dell'Arduino IDE 2.x (URL del pacchetto PJRC qui sopra): installa lo stesso core `teensy:avr`. In entrambi i casi, allo script di build serve solo che il core

sia presente e `arduino-cli` nel PATH.

6 · Le librerie

Il firmware TCERN si compila contro tre **driver personalizzati** che lo adattano al nostro hardware economico. Sono **già nel repo** sotto `teensy/libraries/`, e lo script di build li indica ad `arduino-cli` automaticamente — non devi installarli:

Libreria	Sostituisce	Dove sta la configurazione hardware
i2cEncoderLibV2	i 4 encoder RGB I ² C di TCERN → i nostri 4 encoder meccanici KY-040	ICHOS_ENC_PINS in <code>i2cEncoderLibV2.h</code>
FastTouch	i 3 pad capacitivi di TCERN → i nostri 3 pulsanti tattili momentanei	ICHOS_BTN_PINS in <code>FastTouch.h</code>
IchosOled	aggiunge l' HUD OLED I ² C (lo stato che TCERN mostrava sugli anelli RGB degli encoder)	<code>teensy/libraries/IchosOled/</code>

 **Dove vive l'hardware:** l'assegnazione dei pin sta in quei due header (`ICHOS_ENC_PINS` , `ICHOS_BTN_PINS`) e in `teensy/build_toern.py` — **non** in un `config.h` . (`config.h` appartiene alla vecchia build di fallback NI404 e non viene usato dal port TCERN.)

Lo script di build risolve anche le eventuali librerie standard rimanenti (le **Audio**, **Wire**, **SD**, ecc. incluse nel core Teensy, più qualsiasi cosa nella tua `Arduino/libraries/` utente) come fa normalmente `arduino-cli` .

7 · Come funziona lo script di build

`teensy/build_toern.py` compila lo sketch TCERN **non modificato**, applicando solo le modifiche minime che lo fanno girare sul nostro hardware. Passo per passo:

- Trova i sorgenti.** Cerca lo sketch TCERN sotto `emulator/toern-src/` ; se non c'è, lo **clona** automaticamente dall'upstream (così il repo resta privo di una copia interna).
- Li prepara** in una cartella di build temporanea (gli originali restano intatti).
- Remap dei pin.** I `SWITCH_1/2/3` di TCERN puntano di default a pin che collidono con il nostro KY-040 di destra, quindi vengono rimappati a **25 / 26 / 28** — gli stessi pin di `ICHOS_BTN_PINS` in `FastTouch.h` .
- Snellimento funzioni.** Rimuove la **seconda striscia LED** opzionale (libera un pin e risparmia CPU), come nel nostro build.
- HUD OLED.** Cabla la libreria `IchosOled` (include + `begin()` + una chiamata di rendering su una riga nel `loop()`) così che l'HUD di stato compili come unità di compilazione separata.
- Compila** con `arduino-cli` per l'FQBN qui sotto e copia il risultato in un nome stabile, `toern.hex` .

Il target esatto per cui compila è:

```
teensy:avr:teensy41:usb=serialmidi16,opt=o1std
```

- `usb=serialmidi16` → **USB type Serial + MIDIx16** (così `usbMIDI` funziona).
- `opt=o1std` → `-01`. È il flag critico: il `-02` di default fa crashare il gcc del Teensy sull'enorme unità di compilazione singola di TCERN. Il `-01` compila pulito e il firmware ci sta comunque con margine.

8 · Compilare e caricare

8.1 Compilare

Dalla radice del repo:

```
python teensy/build_toern.py
```

Al termine avrai `teensy/firmware/toern.hex`.

8.2 Compilare e flashare in un passo

Se hai `teensy_loader_cli` nel PATH:

```
python teensy/build_toern.py --flash
```

1. Collega il Teensy 4.1 via USB (cavo **dati**).
2. Il loader di solito programma da solo; se resta in attesa, premi il **pulsantino bianco** sul Teensy.
3. A fine flash parte l'animazione sulla matrice LED.

8.3 Flashare il firmware

Se preferisci flashare un `.hex` **già pronto** (o non hai `teensy_loader_cli`), usa il **flasher GUI a un clic** in `_FLASHER/` — puntalo a `teensy/firmware/toern.hex` e clicca flash.

9 · Compilare dall'Arduino IDE (alternativa)

Puoi anche compilare TCERN a mano nell'Arduino IDE 2.x — utile per studiare il codice passo passo. Devi riprodurre quello che fa lo script:

Voce del menu Tools	Valore
Board	Teensy 4.1
USB Type	Serial + MIDIx16
CPU Speed	600 MHz (default)

Voce del menu Tools	Valore
Optimize	"Fast" ⚠️ (è il -o1 ; il default "Faster"/ -o2 fa crashare il compilatore)
Port	la porta del Teensy (dopo il collegamento)

Dovrai inoltre applicare il remap dei pin (SWITCH_1/2/3 → 25/26/28), rendere visibili all'IDE le tre librerie personalizzate in teensy/libraries/ e cablare a mano l'HUD OLED. **Lo script fa tutto questo per te**, ed è per questo che è il percorso consigliato.

⚠️ Non dimenticare **Optimize** → **"Fast"**. Se lo lasci sul default, la build muore con un internal compiler error (exit 0xffffffff) a metà dell'unico file enorme di TCERN.

10 · Strumenti opzionali (Python, Git)

10.1 Python

Allo script di build serve **Python 3**. (Serve anche Git — vedi sotto — perché lo script clona i sorgenti TCERN alla prima esecuzione.)

Windows: winget install Python.Python.3.12 · **macOS:** brew install python@3.12

i Per lo script di build va bene qualsiasi Python 3 moderno. (Se usi anche il convertitore di campioni wavmaker e lo esegui dai sorgenti su **Python 3.13+**, quello richiede pip install audioop-lts , perché audioop è stato rimosso dalla libreria standard — ma il wavmaker.exe pronto in _SDCARD/ non richiede Python.)

10.2 Git e GitHub CLI

Git è obbligatorio (lo script clona TCERN). GitHub CLI è opzionale.

Windows: winget install Git.Git (+ GitHub.cli opzionale) **macOS:** xcode-select --install (include git) · brew install gh (opzionale)

11 · Checklist e problemi comuni

✓	Controllo
☐	arduino-cli version funziona (è nel PATH)
☐	arduino-cli core list mostra teensy:avr ≥ 1.61.0
☐	Python 3 e Git sono installati
☐	python teensy/build_toern.py produce teensy/firmware/toern.hex

✓	Controllo
☐	(solo build da IDE) USB Type = Serial + MIDIX16 e Optimize = "Fast"
☐	Il Teensy 4.1 ha 16 MB di PSRAM saldati

Sintomo	Causa / soluzione
<code>arduino-cli</code> not on PATH ... dallo script	installa <code>arduino-cli</code> (cap. 4) e riapri il terminale
La compilazione muore con <code>exit 0xffffffff</code> / internal compiler error	hai compilato a <code>-O2</code> ; usa lo script, oppure imposta Optimize → "Fast" (<code>-O1</code>) nell'IDE
<code>teensy:avr</code> mancante / troppo vecchio	esegui <code>arduino-cli core install teensy:avr</code> e controlla <code>core list</code> (serve $\geq 1.61.0$)
Lo script non riesce a clonare i sorgenti TCERN	installa Git, oppure clona a mano in <code>emulator/toern-src/</code>
<code>'usbMIDI' was not declared</code> (build da IDE)	USB Type non su Serial + MIDIX16
<code>teensy_loader_cli</code> not found CON <code>--flash</code>	usa invece il flasher GUI in <code>_FLASHER/</code>
Il Teensy non si programma	premi il pulsante bianco sulla scheda; oppure usa un cavo USB dati , non solo-carica
Porta non visibile	cavo USB solo-carica → usa un cavo dati

Parte del progetto [ichosynth](#) · esegue il firmware reale **TCERN** (SP_) · per il workshop ICHOS 2026.